SPECIFICATION PROPOSAL · RSI-1

Radio signature interchange

A shared library of radio signatures, so an app can recognize the radios another app has already learned.

Proposed by Off Grid Pete LLC
14 November 2026
Document RSI-1, Proposal 1. Supersedes nothing.

Introduction

A radio app can hear a transmitter nearby and still have nothing useful to call it. The broadcast may carry a model string, a protocol name, or a string no person would recognize. Projects that want to name those radios each keep a private list, in a private format. Someone who has built a library in one app cannot give that library to another app and expect the same radios to come up under the same names.

A signature is one entry in such a library. It names a type of radio, and it gives the tests that recognize that type from a broadcast the receiver has already parsed. The tests look at information the radio sends in the clear: its name, a service identifier, a company identifier, part of its address, or a short prefix of the payload. When the tests match, the app can tell the user what type of radio it heard. The match names the type. It does not name a person, and on its own it does not prove which physical unit was heard.

The same signature can carry instructions for reading unencrypted information in that broadcast. A battery level, a status flag, or a short code is often sitting in bytes the radio has already transmitted. The instructions turn those bytes into a label a person can read. Those instructions apply only to bytes the radio has already sent in the clear. The app does not connect to the radio, and it does not decrypt anything. If there is nothing further to read, the signature can still name the radio.

This file is how those libraries move between apps. A person using one of these apps should be able to share a library with someone using another, and the second app should recognize the same radios. A project can publish its own library, under its own name, including radios that another project never hears. Fieldwatch already does something close to this. Releases through 1.1.44 read this kind of library and match the radios in it. Fieldwatch 1.2.0 is the release that will conform to this specification. Its library covers Bluetooth Low Energy and Wi-Fi. The sections below specify the file so that two readers agree on a match, and so that a radio or a field added later does not break a reader that is already in use. How another project takes part is section 26.

> **Status**
>
> This document is a proposal for comment. It is not an adopted standard, and it is not a Bluetooth SIG, IEEE, or IETF publication. Other open-source projects are invited to implement it and to propose a profile for a radio they already receive. Fieldwatch™ is a trademark of Off Grid Pete LLC. The mark is not a registration. Fieldwatch is the first implementation, not the owner of every radio in this file. Releases through 1.1.44 already implement something close to this file. Fieldwatch 1.2.0 will conform to the specification, and it will keep publishing a baseline file that today's phones still load.

1. What this document is

A signature here is a named set of tests against one radio a receiver has already parsed: a name, a service UUID, a company identifier, an address prefix, or a short payload prefix. The file that holds those tests is JSON. This proposal writes down the tests, the file, and the two kinds of parser that read it. Bluetooth Low Energy and Wi-Fi are the radios specified today. Section 17 is how another project adds a radio it hears and this phone does not.

The shape of the file is already in use. Fieldwatch writes it as `fieldwatch-signatures` with `formatVersion` 1, because that is the token phones already accept. A conforming parser also accepts `radio-signatures`, the token for a file written for this proposal. Catalog 113, the stock file current with this proposal, remains a `fieldwatch-signatures` file. This document does not ask anyone to replace that file with a new major version.

Two sorts of statement appear below. Normative statements use the words in section 3 and are the requirements. Informative statements explain why a requirement exists, or describe a parser that is already in the field. Examples and the Fieldwatch release plan are informative except where a paragraph says otherwise.

Contents

Introduction
1. What this document is
2. Scope
3. Requirements language
4. Terms
5. Parsers
6. File format
7. Versions and compatibility
8. Signature identity
9. The observation
10. How a signature matches
11. Rule kinds
12. Names
13. Service UUIDs
14. Addresses and vendor fields
15. Manufacturer data and service data
16. Which radio a rule applies to
17. Other radios
18. Group profile
19. Members that do not affect the match
20. Decode profile
21. Adding a name later
22. Conformance
23. Fieldwatch 1.2.0
24. What a match does not mean
25. Examples
26. Taking part
Appendix A. Test vectors
Appendix B. Baseline registry
References

2. Scope

This specification covers a file of signatures and the decision that one signature does or does not match one observation. A baseline observation is a Bluetooth Low Energy advertisement or a Wi-Fi beacon a receiver has already parsed into the fields in section 9. Other radios are section 17. A file may carry a signature for one of them. A parser matches it only when that parser implements the profile.

The following are out of scope: connecting to a device, reading GATT characteristics, transmitting, estimating distance from signal strength, deciding that a person is present, and any catalog policy that suppresses one matching signature because another also matched. A receiver may apply that kind of policy after the match. It is not part of interchange, and another implementation does not have to copy it to conform.

3. Requirements language

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in capitals.

4. Terms

Term	Meaning
Baseline file	A file that uses formatVersion 1 and only the names in Appendix B. The stock Fieldwatch catalog is a baseline file.
Baseline parser	A parser with the behavior of Fieldwatch releases through 1.1.44. Informative. It defines what a baseline file must avoid.
Conforming parser	A parser that implements the requirements in this proposal. Fieldwatch 1.2.0 is intended to be the first Fieldwatch release that conforms. Releases through 1.1.44 are close, and they are the baseline parser.
Publisher	The party that assigned a signature id. Optional in the file. Absent on every signature published before this proposal.
Radio profile	The fields and rule kinds defined for one radio token. BLE and WIFI are specified. Further profiles are optional and empty until an implementer who receives that radio writes them. Section 17.
Match	The result that a signature's rules accept an observation. A match names a pattern. It does not by itself name a person or prove a model.

5. Parsers

A parser is the program that reads the JSON and decides which signatures match an observation. The file format and the parser are specified together because a file that is safe for one parser can be unreadable to another. Fieldwatch releases through 1.1.44 already have a parser in the field, and that parser is close to this proposal. This proposal does not change that parser. It defines the parser Fieldwatch 1.2.0 will use, and it constrains future files so the parser already installed keeps working.

5.1 What every parser already has in common

A baseline parser and a conforming parser both accept JSON as defined in RFC 8259, encoded in UTF-8. Both accept a leading Unicode byte order mark and ignore it. Both ignore object members they do not recognize. Both apply default values where this document gives a default, so a file written before a member existed still loads.

Ignoring an unknown member is what makes an added field safe. The member is in the file for a parser that knows it. A parser that does not know it skips the member and matches on the members it does know. This is already how Fieldwatch reads the file.

5.2 The baseline parser

The baseline parser is informative. It is the parser in Fieldwatch releases through 1.1.44, and it is the reason the public catalog cannot yet contain every idea in this proposal.

- It accepts `formatVersion` 1. If `formatVersion` is greater than 1, it rejects the whole file and asks for a newer app. It does not apply any signature from that file.
- If a rule kind, a radio value, a signature class, or a decode type is a string the parser does not know, it rejects the whole file the same way. One unrecognized name on one signature is enough.
- If a decode source is a string it does not know, it drops that decode map and keeps the signature. Matching is unchanged. This is the one unknown name the baseline parser already softens.
- It identifies a signature by `id` alone. It does not store `publisher`.
- A stock update whose `catalogVersion` is less than or equal to the version already stored is ignored. A higher catalog version is applied. Unknown members have already been dropped by then, so they are not saved.

5.3 The conforming parser

A conforming parser MUST do all of the following.

- Accept every baseline file and produce the same matches as sections 10 through 16 for every signature that uses only baseline rule kinds.
- Ignore unknown object members. Where this proposal defines a member, store it. Where a member is absent, apply the default in section 6.
- On `formatVersion` 1, an unknown rule kind or an unknown radio token MUST NOT match. The parser MUST read the rest of the file and MUST NOT reject the file. If `matchAny` is true, the parser ignores that rule and evaluates the others. If `matchAny` is false, the signature MUST NOT match, because the unknown rule is a required test the parser cannot perform. If no enabled rule remains that the parser understands, the signature MUST NOT match. Section 17 is the same rule for a radio the parser does not implement.
- An unknown signature class MUST NOT prevent a match. The parser SHOULD keep the raw class string. If it cannot, it MUST treat the class as `OTHER` for display only.
- An unknown decode type MUST cause that field alone to be omitted. An unknown decode source MUST cause the whole decode object to be omitted. In both cases the signature MUST still match on its rules.
- If `formatVersion` is greater than the highest version the parser implements, the parser SHOULD still read the file, apply every member it understands, ignore the rest, and report that the file is newer than the parser. It SHOULD NOT refuse the file only because the number is higher.
- Identity MUST follow section 8. Adding `publisher` to a signature whose stored copy has no publisher MUST NOT create a second signature.

The last requirement is the one that protects a phone already running 1.1.44. That phone never stores `publisher`. Fieldwatch 1.2.0 will see a stock file that has the field and a phone copy that does not. Those two copies are the same signature.

5.4 What a publisher of a file owes each parser

If the file will be read by	The file MUST
A baseline parser, including Fieldwatch 1.1.44	Stay a baseline file: formatVersion 1, and only names from Appendix B. It MAY add object members those parsers will ignore, including publisher.
Only conforming parsers	Use formatVersion 1 for any addition that an older match would still get right by ignoring the new member. Raise formatVersion only for a change described in section 7.

> **Why the public catalog stays on format 1**
>
> The stock file at fieldwatch-signatures-v2.json is what current phones download. A formatVersion above 1, or one rule kind or radio token those phones do not know, makes Update stock catalog refuse the entire file. New members that those phones ignore are safe to add only in a release whose parser stores them. Until that release is the one on the phone, a catalog number spent on an ignored field is stored, and a later file with the same number is skipped. Section 23 is the order that avoids that.

6. File format

A file is one JSON object. A conforming parser MUST accept the members in this section. A writer MUST emit UTF-8 JSON. A writer SHOULD emit `format` and `formatVersion`. Integers MUST be within the range that can be represented exactly in IEEE 754 binary64, which covers every field in this document.

6.1 Top-level members

Member	Requirement
`format`	String. A file a baseline parser must read MUST be fieldwatch-signatures. A Fieldwatch parser also accepts the historical name spectre-signatures. Other parsers MAY reject that name. A file written only for conforming parsers MAY be radio-signatures. A conforming parser MUST accept both fieldwatch-signatures and radio-signatures.
`formatVersion`	Integer. MUST be 1 or greater. The baseline and this proposal's additions are version 1. Absent means 1.
`catalogVersion`	Integer. Optional. Identifies one publisher's snapshot of a stock catalog. It is not a schema version. Absent means 0. A consumer MUST NOT interpret it as formatVersion.
`exportedAt`	String. Optional. When the file was written, for humans. Not used in matching.
`appVersion`	String. Optional. The application that wrote the file. Not used in matching.
`publisher`	String. Optional. The publisher of this file as a whole, using the token in section 8. A signature may name a different publisher.
`fleets`	Array of signature objects. The name fleets is historical and is retained so existing files parse. The array MUST be present. An empty array is a valid file with nothing to match.

6.2 Signature object

Each element of `fleets` is an object. Members that do not affect the match are still part of the file so an application can round-trip a catalog. They are specified in section 19.

Member	Requirement
`id`	String. REQUIRED. Stable key assigned by the publisher. MUST be non-empty. MUST NOT change for a published signature. The Fieldwatch stock ids in use today remain the ids.
`publisher`	String. Optional. Token from section 8. Absent means unspecified.
`name`	String. REQUIRED. Human label. Not used in matching.
`rules`	Array of rule objects. REQUIRED. The array may be empty only for a group signature under section 18. Otherwise an empty rules array matches nothing.
`matchAny`	Boolean. Default true. Section 10.
`kind`	String. Signature class. Default OTHER. Does not affect the match. Appendix B lists the baseline classes.

6.3 Rule object

Member	Requirement
`kind`	String. REQUIRED. One of the names in section 11, or a name added under section 21.
`text`	String. Default empty. Meaning depends on kind.
`companyId`	Integer. Default 0. A Bluetooth company identifier, written in decimal. The same number as the 16-bit identifier assigned by the Bluetooth SIG.
`dataPrefixHex`	String. Default empty. Hexadecimal payload prefix. Separators and case are ignored.
`radio`	String or null. A radio token. Baseline values are BLE, WIFI, or null. Default null. JSON null and an omitted member are the same. Section 16. Any other token is a profile under section 17, and a baseline file MUST NOT use it.
`enabled`	Boolean. Default true. A rule with enabled false MUST NOT match and MUST NOT count as a remaining rule under matchAny false.

7. Versions and compatibility

Two integers travel in the file, and they answer different questions. `formatVersion` is the schema. `catalogVersion` is one publisher's revision of the signatures inside a stock file. Fieldwatch catalog 113 and formatVersion 1 are both current as of this proposal. Another publisher uses its own catalog number and the same format version.

A change falls into one of three classes.

- **Ignored by an older parser.** A new optional member with a default, such as publisher. formatVersion stays 1. A baseline parser loads the file and matches as before. A conforming parser stores the member.
- **A new registered name** (a rule kind, a radio token, a class, a decode type). formatVersion stays 1. A conforming parser skips the piece it does not know, as in section 5.3. An unknown radio token on a signature with `matchAny` false fails that signature closed, under section 17. A baseline parser rejects the whole file. A file that baseline parsers must read MUST NOT use the new name.
- **A change an older matcher would get wrong** if it applied the rules it knows and ignored the rest. That change MUST increase formatVersion. An example would be a new rule kind that is defined to exclude observations which the older rules would have accepted. Skipping that rule would label the wrong radios. A baseline parser will reject the file. A conforming parser built before the new version reports that the file is newer and applies only what it understands, and it MUST NOT present that partial result as complete.

Writers MUST NOT increase formatVersion for a change in the first class. Doing so turns a compatible file into a file the baseline parser refuses.

8. Signature identity

The id is the stable name of a signature. Fieldwatch stock ids look like `fleet-samsung-tv`. Those strings are in use on phones, including mute, extra rules a person added, and bookmarks. This proposal does not rename them.

A publisher token lets two parties use the same id text without becoming the same signature. The token MUST match `[a-z][a-z0-9.-]{0,63}`. The token for signatures assigned by this project is `fieldwatch`. Another publisher SHOULD use a domain name it controls, in lowercase, such as `example.com`.

Two signature objects are the same signature when all of the following hold.

- The id strings are equal, compared as Unicode code points, case-sensitive.
- The publishers are compatible. They are compatible when they are equal, or when either publisher is absent.

If both publishers are present and they differ, the signatures are different even when the id text is the same. A consumer MUST keep both. A consumer MUST NOT satisfy this rule by deleting the publisher and comparing ids alone. That would merge two parties' work.

A writer that did not assign a signature MUST preserve the publisher it read, including preserving absence. A writer MAY set `publisher` to its own token on a signature it assigns. Fieldwatch MAY set `fieldwatch` on a stock id it assigned when the stored publisher is absent. It MUST NOT set that token on a custom signature.

> **Same id, new field, one row**
>
> A phone stored fleet-samsung-tv with no publisher. A later file has id fleet-samsung-tv and publisher fieldwatch. The publishers are compatible because one is absent. The update replaces the stock rules and keeps the phone's mute and any extra rules. It does not add a second Samsung TV signature.

9. The observation

Matching takes an observation the receiver has already parsed. A conforming matcher MUST be able to read the following. It MUST NOT require signal strength, position, or a raw frame beyond the bytes a decode profile needs.

Field	Use
Radio	BLE or WIFI for a baseline observation. A profile in section 17 may name another radio and the fields that radio provides. Those fields are not part of a baseline observation.
Address	48-bit MAC-48 address, as transmitted.
Name	Advertised name or SSID. Empty if the frame has none. A blank name is not a match for a name rule.
Hidden SSID	True when a Wi-Fi beacon sets the hidden-SSID flag.
Service UUIDs	UUIDs advertised as services.
Service data	Pairs of UUID and payload bytes.
Manufacturer records	Pairs of company identifier and payload bytes. The company identifier is the 16-bit Bluetooth value.
Vendor IE OUIs	Organizationally unique identifiers carried in Wi-Fi vendor-specific information elements.
Last heard	Used only by the group profile.

The fields in the table are the baseline fields. A profile added under section 17 names any further fields that radio provides, and a core matcher of baseline files does not have to read them. Hexadecimal strings in rules and in observations are compared after removing separators (colon, hyphen, space) and after converting letters to uppercase. A leading `0x` is not required and MUST be ignored if a writer included it on an address. Company identifiers are numbers, not hex strings.

10. How a signature matches

Only enabled rules are considered. A signature with no remaining enabled rules does not match, unless section 18 says it is a group signature with an empty rule list. A rule the parser cannot perform is not a remaining rule that passed. Section 5.3 says when that fails the signature.

If `matchAny` is true, the signature matches when any remaining rule matches. If `matchAny` is false, the signature matches when every remaining rule matches. A signature has no third mode. A writer that needs both a required rule and a list of alternatives MUST use two signatures.

Each signature is tested on its own. A match on one signature MUST NOT, by this specification, remove a match on another. An application may hide a label afterward. That step is outside conformance.

Rule evaluation order MUST NOT change the result. A matcher MAY stop early when `matchAny` is true and one rule has already matched. The set of matching signatures MUST be the same either way.

11. Rule kinds

The baseline registry has eleven kinds. A conforming parser MUST implement each of them. The subsections give the match. Section 16 further restricts the match by radio.

Kind	What it tests
`NAME_CONTAINS`	The name contains text, case-insensitive. An empty name does not match. An empty text does not match.
`NAME_GLOB`	The whole name matches the pattern in section 12.
`SERVICE_UUID`	An advertised service UUID or a service-data UUID is equivalent under section 13.
`SERVICE_DATA`	A service-data record matches section 15.

Kind	What it tests
MANUFACTURER_ID	A manufacturer record has companyId equal to the rule's companyId. Zero matches company 0 only, not every company.
MANUFACTURER_DATA	A manufacturer payload starts with dataPrefixHex. Section 15.
OUI	The address, and on Wi-Fi the vendor IE, starts with the prefix. Section 14.
MAC_PREFIX	The address starts with the prefix. Vendor IEs are not tested.
VENDOR_IE_OUI	A Wi-Fi vendor IE starts with the prefix. The address is not tested.
RADIO_KIND	The observation's radio equals the rule's radio. A null radio matches either baseline radio, BLE or WIFI. It does not match a radio added under section 17.
HIDDEN_SSID	The observation is Wi-Fi and the hidden-SSID flag is set. text is ignored.

12. Names

Name comparison uses the advertised name as a character string. It is case-insensitive under Unicode simple case folding for the letters A to Z, which is the comparison Fieldwatch implements with an ignore-case match. A name rule MUST NOT match when the advertised name is empty or missing.

NAME_CONTAINS matches when text occurs as a substring. NAME_GLOB matches the entire name. The pattern is interpreted as follows, and then anchored at both ends so a pattern without a wildcard is an exact name.

- * matches any sequence of characters, including the empty sequence.
- ? matches exactly one character.
- Every other character matches itself. Parentheses, brackets, dots, and hyphens are literal. They are not regular-expression syntax.
- A pattern that is empty MUST NOT match.

So TD-H9* matches TD-H9 and TD-H9-0536, and does not match XTD-H9. ASB?????? matches exactly six characters after ASB. ICOM BT(IC-705) matches that name only, including the parentheses. walkie-talkie does not match walkie-talkie 2 or walkie talkie.

13. Service UUIDs

UUID text is compared case-insensitively after hyphens are removed. A 16-bit Bluetooth UUID, a 32-bit Bluetooth UUID, and a 128-bit UUID on the Bluetooth base are the same identifier and MUST match each other. The Bluetooth base tail is the 96 bits 0000-1000-8000-00805F9B34FB.

- Four hex digits XXXX match XXXX, 0xXXXX, 0000XXXX, and 0000XXXX-0000-1000-8000-00805F9B34FB.
- Eight hex digits match that 32-bit value on the Bluetooth base. They match a 16-bit short form only when the high half is 0000.
- A 128-bit value matches a shorter form only when it ends in the Bluetooth base tail, and the 16-bit short form only when it also begins with 0000.
- Any other 128-bit value MUST be stored in full. A writer MUST NOT shorten it to four or eight hex digits. A parser MUST NOT treat it as equal to a different UUID that shares those digits.

A SERVICE_UUID rule looks at advertised service UUIDs and at the UUID of each service-data record. It does not look at GATT characteristics that were not advertised. The KISS TNC UUID 00000001-ba2a-46c9-ae49-01b0961f68bb does not end in the Bluetooth base tail, so it matches only that 128-bit value, not the 16-bit value 0001.

14. Addresses and vendor fields

An address prefix matches when the address hex starts with the rule prefix. A 24-bit OUI is six hex digits. A longer prefix is a longer address match. Separators in the rule are ignored, so `00:14:7B` and `00147B` are the same prefix.

`OUI` also matches a Wi-Fi vendor IE whose hex starts with the prefix. Two protocol OUIs MUST NOT match through a vendor IE, because they mark WPA and RSN rather than a product: `0050F2` and `000FAC`. They MAY still match an address, if a rule names that prefix and the address starts with it.

On Wi-Fi, a 24-bit `OUI` rule also matches an individually addressed, locally administered BSSID after the local bit (bit 1 of the first octet) is cleared. Guest and extra SSIDs often set that bit on the burned-in vendor prefix. The matcher MUST NOT clear the bit on a group address. `MAC_PREFIX` and `VENDOR_IE_OUI` do not use this recovery.

A prefix of a length other than six hex digits matches only by a hex prefix on the address or, for `OUI` and `VENDOR_IE_OUI`, on a vendor IE. It does not use the 24-bit local-bit recovery.

15. Manufacturer data and service data

`MANUFACTURER_DATA` requires a non-empty `dataPrefixHex`. A record matches when its payload hex starts with that prefix. If `companyId` is 0, any company identifier is accepted. If it is any other value, the record's company identifier MUST be equal to it.

`SERVICE_DATA` uses `text` as a UUID and `dataPrefixHex` as payload bytes. If the UUID is present, the record UUID MUST be equivalent under section 13, and the payload MUST start with the prefix. An empty prefix accepts any payload. If the UUID is empty and the prefix is present, the rule matches a record whose payload contains that prefix or the same bytes reversed in pairs. If both are empty, the rule MUST NOT match.

16. Which radio a rule applies to

When `radio` is `BLE` or `WIFI`, the rule MUST be evaluated only against that radio. A rule that does not apply does not match, and under `matchAny` false it is ignored rather than counted as a failure. That exception is only for a radio token the parser implements, when the observation is a different radio the parser also implements. It is how one signature carries a Bluetooth name and a Wi-Fi name as alternatives. An unknown radio token is not this case. Section 17 applies, and under `matchAny` false the signature does not match.

When `radio` is null or omitted, the rule's kind decides among the baseline radios only. The omission MUST NOT expand onto a radio token added later.

- Name rules, `OUI`, and `MAC_PREFIX` apply to both baseline radios, BLE and WIFI. They do not apply to a later radio token.
- `SERVICE_UUID`, `SERVICE_DATA`, `MANUFACTURER_ID`, and `MANUFACTURER_DATA` apply to Bluetooth Low Energy only. These fields are Low Energy advertisements.
- `VENDOR_IE_OUI` and `HIDDEN_SSID` apply to Wi-Fi only.
- `RADIO_KIND` with a null radio matches either baseline radio and does not match a later token. The rule does not constrain anything among baseline radios. Writers SHOULD omit such a rule.

Fieldwatch releases through 1.1.44 already scope a signature to Bluetooth when every rule is Bluetooth-scoped, including a service or company rule whose radio member was omitted. On a signature that also contains a Wi-Fi rule, those releases can still test an omitted-radio service rule against Wi-Fi. A conforming parser MUST apply the list above even on a mixed signature. The stock catalog sets `radio` explicitly on new service and company rules, so baseline files do not depend on the mixed case.

A signature for Classic Bluetooth, or for any radio added later, MUST set `radio` on each rule that should match that radio. An omitted-radio name rule MUST NOT match a Classic inquiry that advertises the same name.

17. Other radios

The baseline radios are Bluetooth Low Energy and Wi-Fi. Other receivers hear more. Kismet, for example, reports Bluetooth Classic inquiry, IEEE 802.15.4, and 433 MHz sensor broadcasts in addition to Wi-Fi and Low Energy. This file can carry signatures for those radios. A parser matches a rule only for a profile it implements. Fieldwatch implements BLE and Wi-Fi only, and this proposal does not require it to match the others.

A radio token is an open name, registered the same way as a rule kind under section 21. The baseline tokens are `BLE` and `WIFI`. A new token is written into a revision of this document together with the observation fields that radio provides.

A profile is the set of fields and rule kinds defined for one radio token. A product claims the profiles it implements. The BLE and Wi-Fi profiles are enough for a core matcher of baseline files. A profile for another radio, such as Classic Bluetooth or 802.15.4, is OPTIONAL. Naming an example token in this document does not register the profile.

An observation for a radio the parser does not implement MUST NOT match any signature on the strength of rules the parser skipped.

- A rule whose radio token the parser does not implement MUST NOT match.
- If `matchAny` is true, the parser ignores that rule and evaluates the others.
- If `matchAny` is false, the signature MUST NOT match. The unknown rule is a required test the parser cannot perform.

`NAME_CONTAINS`, `NAME_GLOB`, `OUI`, and `MAC_PREFIX` need only a name or an address. A profile whose observation has those fields MAY use them. `SERVICE_UUID`, manufacturer data, vendor information elements, and hidden SSID stay with the radios that have those fields. A test those kinds cannot state, such as a Bluetooth Classic class of device, is a new rule kind added with that profile.

This proposal does not define Classic Bluetooth, 802.15.4, or 433 MHz sensor fields. Those profiles are empty until an implementer who receives them writes the fields, the rule kinds, and the test vectors. An empty profile is not a contribution. A file that baseline parsers must read MUST NOT use a radio token other than `BLE`, `WIFI`, or null.

18. Group profile

The group profile is OPTIONAL. A core matcher MUST implement sections 10 through 16 and MUST NOT report a match that depends only on this section. Fieldwatch implements the profile. A signature uses it when `minPeers` is greater than 0, or `clusterByOui` is true, or `sequentialMac` is true.

The window is `peerWindowSec` seconds, or 60 when that member is 0 or absent. Only observations last heard inside the window are peers. The required count is `minPeers`, or 3 when the signature is in this profile and `minPeers` is 0.

An observation is eligible when its signature rules match, or when the rule list is empty, or when `clusterByOui` is true and the name is blank or the address is randomized. Peers are other eligible observations in the window. When `clusterByOui` is true, a peer MUST share the first 24 bits of the address. When `sequentialMac` is true, the absolute difference of the last 16 bits of the two addresses MUST be 64 or less. The signature matches an eligible observation when the peer count, including itself, reaches the required count.

The stock Fieldwatch catalog at the date of this proposal contains no group signature. A baseline file can omit the four members. The defaults are `minPeers` 0, `peerWindowSec` 60, and both flags false, which means the signature is not in this profile.

19. Members that do not affect the match

A conforming parser that stores and rewrites catalogs MUST preserve these members. A matcher that only reports matches MAY ignore them. None of them change sections 10 through 16, and none of them change the group test in section 18.

Member	Meaning
`enabled`	Boolean, default true. The application is willing to use the signature. A stock update SHOULD preserve a local value of false, which is how a person mutes one stock signature.
`colorIndex`	Integer, default 0. An application palette slot, not a color value. Fieldwatch uses a fixed palette. Other applications MAY ignore it.
`notes`	String, default empty. Explanation for a person reading the signature. Not a match key.
`attentionNote`	String, default empty. Application caution text. Empty means the signature asks for no extra attention. Fieldwatch does not beep a signature whose attention note is empty, except a local policy for one class. That policy is not part of this file.
`builtIn`	Boolean, default false. True when the signature came from the application's stock catalog. A stock update uses it together with the id.
`kind`	Class token. Display and filtering. Baseline tokens are in Appendix B. Unknown tokens follow section 5.3.

A consumer that overlays a stock file onto a phone MUST keep, for the same signature under section 8, the local `enabled` value and any local rules that the incoming signature does not already contain. It MUST NOT clear bookmarks or other application settings as a side effect of reading the file.

20. Decode profile

The decode profile is OPTIONAL. It maps bytes that already matched into labeled numbers and text for display. It MUST NOT change whether the signature matches. A core matcher MAY ignore the `decode` member. Fieldwatch implements the profile. Omitting `decode` means the signature has no field map.

The object has `source` (`manufacturerData` or `serviceData`), an optional `companyId`, an optional `serviceUuid`, an optional `includeCompanyId` boolean, and `fields`, an array of field objects. When `includeCompanyId` is true, the two-byte little-endian company identifier is prepended to the manufacturer payload before offsets are applied.

A field has `id`, `label`, `offset` (zero-based), optional `length`, `type`, and `endian` (`le` or `be`, default `le`). Optional `bitOffset` and `bitWidth` apply to type `bits`. Optional `modulo` is applied to the raw numeric value before scale. The displayed number is raw × scale + offsetAdd, where a missing scale is 1 and a missing offsetAdd is 0. Optional `unit`, `enum` (a map from the raw key to a label), and `enumNotes` are display. Optional `live` and `liveEmphasis` are application hints for a live row. Optional `when` is a gate.

A gate has `offset`, `length` (default 1), `op`, and `valueHex`. The operations are `eq`, `neq`, `mask` (every 1-bit in the value is set in the payload), `nmask` (every 1-bit in the value is clear), and `len` (the payload length equals `length`; offset and value are ignored). Optional `and` is a second gate; both MUST pass. A field whose gate fails MUST be omitted from the display. The signature still matches.

Baseline numeric types and their default widths are:

Type	Width and reading
`u8 i8 bool`	1 byte. bool is 0 or non-zero. i8 is signed.

Type	Width and reading
`u16 i16`	2 bytes, endian as given.
`u24`	3 bytes, endian as given.
`u32 i32 f32`	4 bytes. f32 is IEEE 754 binary32.
`mac`	6 bytes, displayed as an address.
`bits`	The bit field given by bitOffset and bitWidth, reading across bytes as needed.
`utf8 hex`	length bytes. Default length 1 when length is omitted. utf8 is text. hex is hexadecimal.

An unknown `source`, `type`, `endian`, or gate `op` follows section 5.3. The signature's rules still match. A baseline parser rejects the file on an unknown type, endian, or gate operation, and already drops an unknown source.

21. Adding a name later

A new rule kind, radio token, class, decode type, endian, or gate operation is added by writing it into a revision of this document, with the match or the reading specified tightly enough that two parsers agree. A radio token is added with the profile in section 17, written by an implementer who receives that radio. The revision keeps formatVersion 1 unless the new name would make a parser that skips it label a different radio than a parser that understands it.

Until a name is in the revision a parser implements, that parser treats it as unknown under section 5.3. Publishers that still need baseline parsers to read the file wait, and express the signature with names those parsers know. A tighter name glob is the usual way. A new kind is reserved for a test the existing kinds cannot state.

Class tokens are a vocabulary for display. Adding one does not change matching. The cost of adding one is only that a baseline parser rejects a file that contains it. A file for those parsers keeps using a class they know, or `OTHER`.

22. Conformance

An implementation claims one or more of the following.

- **Core matcher.** Given a baseline file and an observation on BLE or WIFI, it reports the same matching signature ids as sections 10 through 16. It may ignore group signatures, decode, application members, and any radio profile it does not implement.
- **Conforming parser.** It meets section 5.3, reads and writes the file in section 6, and includes a core matcher. If it stores catalogs, it preserves section 19 and identity in section 8. It MUST accept `fieldwatch-signatures` and `radio-signatures`.
- **Group profile.** It implements section 18 in addition to the core.
- **Decode profile.** It implements section 20. Failures inside a decode MUST NOT remove a match the rules produced.
- **Additional radio profile.** OPTIONAL. It implements one radio token registered under section 17, and it states which token. A product that does not implement the token leaves those rules unmatched, as section 5.3 requires. Fieldwatch claims the BLE and Wi-Fi profiles only.

Appendix A is the start of the conformance set. An implementation that disagrees with a vector is not conforming, even if the surrounding prose can be read another way. Where this document and a later agreed test set disagree, the test set is corrected by a revision of this document, not by silent drift.

A baseline parser is not a conforming parser of this proposal. It is the installed base. Files intended for it follow section 5.4. No claim of conformance is withdrawn from Fieldwatch 1.1.44, because that release predates this document and correctly reads the baseline file.

23. Fieldwatch 1.2.0

Fieldwatch releases through 1.1.44 implement something close to this proposal. They read a baseline file and match the signatures in it. They are not yet conforming parsers. Fieldwatch 1.2.0 is the release that will conform. This section is the plan for that release. It binds 1.2.0 only in the sense that 1.2.0 is how this project will meet the conforming parser. Other products are not required to ship on the same day.

- Fieldwatch 1.2.0 adds the conforming parser in section 5.3. Behavior on a baseline file stays the same for every stock signature that exists at Proposal 1.
- The stock file stays `format` `fieldwatch-signatures` and `formatVersion` 1. It uses no radio token other than `BLE`, `WIFI`, or null.
- Each stock signature gains `publisher` `fieldwatch`. Ids are unchanged. The catalog number increases in that same release, so a phone whose parser can store the field will apply it. The field is omitted when empty, so custom signatures without a publisher stay that way.
- A phone that updates from an older app version treats a stored row with no publisher and a file row with publisher `fieldwatch` as the same signature when the ids are equal. Mute and extra rules remain.
- The public file does not contain a rule kind, class, radio, or decode type that 1.1.44 does not know. Those phones still update. The conforming parser implements the unknown-radio rule in section 5.3, so a later file from another project can name a radio this phone does not match without being rejected.
- Appendix A is added to the project's tests. A change to matching that fails a vector is a format change under section 7, not an unannounced improvement.

Publishing the publisher field before the release that stores it would move older phones onto a new catalog number and then throw the field away. The following file, still at that number, would be skipped. This plan avoids that by shipping the parser and the field together.

24. What a match does not mean

A match means the observation satisfied the rules. Names are chosen by the device and can be changed. Company identifiers are shared by many products from one company. An address prefix is shared by every interface that vendor shipped. A service UUID is often shared by a family, and sometimes by every device that speaks a published protocol.

A conforming application MUST NOT present a match, by itself, as identification of a person, as proof that a medical or safety event occurred, or as proof that a particular object is the one a rule was written about. Notes in the Fieldwatch catalog say this for health and finder signatures. The requirement belongs on the application, because the file cannot enforce a sentence.

The file contains no location, no owner, and no credential. A publisher SHOULD NOT put a serial number, a full address, or a person's name into a signature id, a note, or a pattern, beyond the model-level names the rule needs.

25. Examples

A baseline file. A Fieldwatch 1.1.44 parser and a conforming parser both match a Bluetooth advertisement named TD-H9-0536, and both refuse a Wi-Fi network of that name because the rule sets the radio.

```
{
  "format": "fieldwatch-signatures",
  "formatVersion": 1,
  "catalogVersion": 113,
  "fleets": [{
    "id": "fleet-tidradio-td-h9",
    "name": "TIDRADIO TD-H9",
    "matchAny": true,
    "kind": "TWO_WAY",
    "rules": [
      { "kind": "NAME_GLOB", "text": "TD-H9*", "radio": "BLE" }
    ]
  }]
}
```

The same signature, once Fieldwatch 1.2.0 adds the publisher. A 1.1.44 parser ignores `publisher` and matches as before. A conforming parser stores the publisher and treats this row as the same signature as the one above.

```
{
  "id": "fleet-tidradio-td-h9",
  "publisher": "fieldwatch",
  "name": "TIDRADIO TD-H9",
  "matchAny": true,
  "kind": "TWO_WAY",
  "rules": [
    { "kind": "NAME_GLOB", "text": "TD-H9*", "radio": "BLE" }
  ]
}
```

A file a conforming parser accepts and a baseline parser rejects. `matchAny` is true, so the unknown rule is ignored and the name rule still matches. The same signature with `matchAny` false MUST NOT match, because the unknown rule is required and the parser cannot perform it. The parser does not reject the file.

```
{
  "id": "example.beacon",
  "publisher": "example.com",
  "name": "Example beacon",
  "matchAny": true,
  "rules": [
    { "kind": "NAME_GLOB", "text": "EX-BEACON", "radio": "BLE" },
    { "kind": "SOMETHING_NEW", "text": "later" }
  ]
}
```

This id is not a Fieldwatch stock id, and the publisher is present on both sides of any later merge, so it will not attach itself to a Fieldwatch row that happens to use the same id text.

A signature for a radio Fieldwatch does not hear. The rule sets `radio`, so an omitted-radio name of the same text does not match this observation. A parser with no Classic profile does not match the signature. A file that uses the token MAY set `format` to `radio-signatures`. It is not a baseline file, and a 1.1.44 parser would reject it. The Classic profile itself is not defined here.

```
{
  "format": "radio-signatures",
  "formatVersion": 1,
  "fleets": [{
    "id": "example.classic-handset",
    "publisher": "example.com",
    "name": "Example Classic handset",
    "matchAny": true,
    "rules": [
      { "kind": "NAME_GLOB", "text": "HANDSET-*", "radio": "BT_CLASSIC" }
    ]
  }]
}
```

26. Taking part

This document is a proposal from Off Grid Pete LLC, opened so other open-source projects can build receivers that read the same file. Fieldwatch is the first implementation and the publisher of the baseline profiles for Bluetooth Low Energy and Wi-Fi. Releases through 1.1.44 are close to this file. Fieldwatch 1.2.0 will conform to it. Fieldwatch does not have to be the only implementation.

A project takes part by implementing the core matcher and stating which profiles it implements, by reviewing this proposal, or by proposing a profile for a radio that project already receives. The proposal includes the observation fields, the rule kinds, and the test vectors. The profile should be written by someone who can hear that radio. An empty profile is not a contribution.

Comments and proposed profiles are invited at github.com/OffGridPete/Fieldwatch. A revision will name projects that contribute a profile. There is no fee and no separate organization in this proposal. If the set of implementers grows, a later revision can describe how changes are agreed. This revision does not create that body in advance.

Fieldwatch will keep publishing a baseline file that current phones load. Another project can publish its own file, with its own publisher token, including radios Fieldwatch does not match. A conforming parser reads the signatures it understands and leaves the rest unmatched.

Appendix A. Test vectors

Each row is one observation against one rule, unless a signature is named. Names are the advertised name. The radio is Bluetooth Low Energy unless the row says otherwise. A blank cell means the member is omitted.

Rule	Observation	Match
NAME_GLOB TD-H9*, radio BLE	Name TD-H9-0536	Yes
NAME_GLOB TD-H9*, radio BLE	Name td-h9	Yes
NAME_GLOB TD-H9*, radio BLE	Name XTD-H9	No
NAME_GLOB TD-H9*, radio BLE	Wi-Fi name TD-H9-0536	No
NAME_GLOB TD-H9*, radio BLE	Empty name	No
NAME_GLOB ASB??????, radio BLE	Name ASB99C2F0	Yes
NAME_GLOB ASB??????, radio BLE	Name ASB99C2F	No
NAME_GLOB ASB??????, radio BLE	Name ASB99C2F01	No
NAME_GLOB walkie-talkie, radio BLE	Name Walkie-Talkie	Yes
NAME_GLOB walkie-talkie, radio BLE	Name walkie-talkie 2	No

Rule	Observation	Match
NAME_GLOB ICOM BT(IC-705), radio BLE	Name ICOM BT(IC-705)	Yes
NAME_GLOB IC-705, radio WIFI	Wi-Fi name IC-705	Yes
NAME_GLOB IC-705, radio WIFI	Wi-Fi name IC-7050	No
NAME_GLOB IC-705, radio WIFI	Bluetooth name IC-705	No
SERVICE_UUID FEA0, radio omitted	Service 0000FEA0-0000-1000-8000-00805F9B34FB	Yes
SERVICE_UUID FEA0, radio omitted	Wi-Fi, that UUID presented as a service	No
SERVICE_UUID 00000001-ba2a-46c9-ae49-01b0961f68bb	That UUID, any case	Yes
SERVICE_UUID 00000001-ba2a-46c9-ae49-01b0961f68bb	The same UUID with 0002 in the first group	No
SERVICE_UUID 00000001-ba2a-46c9-ae49-01b0961f68bb	16-bit UUID 0001	No
MANUFACTURER_ID companyId 1704, radio BLE	Company 1704	Yes
MANUFACTURER_ID companyId 1704, radio BLE	Company 117	No
OUI 00147B	Address 00:14:7B:01:02:03	Yes
OUI 0050F2	Wi-Fi vendor IE 00:50:F2 only, other address	No
matchAny false, two name rules	A name that meets only one	No
Unknown kind plus a matching name rule, matchAny true	The name matches	Yes on a conforming parser. The baseline parser rejects the file.
Unknown kind plus a matching name rule, matchAny false	The name matches	No on a conforming parser. The unknown rule is required. The baseline parser rejects the file.
NAME_GLOB TD-H9*, radio omitted	Bluetooth Classic inquiry named TD-H9	No. An omitted radio is BLE and WIFI only.
radio BT_CLASSIC, matchAny true, plus a matching BLE name rule. Parser has no Classic profile	A BLE name that meets the BLE rule	Yes. The Classic rule does not match. The BLE rule does.
radio BT_CLASSIC, matchAny false. Parser has no Classic profile	A Classic inquiry the rule would have matched	No. The parser cannot perform the required test, so the signature does not match.
publisher fieldwatch, id fleet-tidradio-td-h9	Stored copy of that id with no publisher	Same signature, not a second row
publisher example.com, id fleet-tidradio-td-h9	Stored fieldwatch row with that id	Different signature

Appendix B. Baseline registry

These are the names a baseline parser accepts. A file that uses only these names, with formatVersion 1, is a baseline file. The class list is the Fieldwatch set at Proposal 1, including values kept so older files still parse.

Rule kinds. OUI, MAC_PREFIX, NAME_CONTAINS, NAME_GLOB, SERVICE_UUID, SERVICE_DATA, MANUFACTURER_ID, MANUFACTURER_DATA, RADIO_KIND, HIDDEN_SSID, VENDOR_IE_OUI.

Radios. Baseline tokens are BLE, WIFI, or null. A further token is registered with a profile under section 17 and is not a baseline name. A file that baseline parsers must read MUST NOT use it. This proposal registers no further

token. BT_CLASSIC in the examples is an illustration, not a registered profile.

Classes. FINDER, BEACON, SIGNAGE, WEARABLE, SURVEILLANCE, DRONE, HACKING, BODYWORN, LAW_ENFORCEMENT, VEHICLE, GLASSES, AUDIO, CAMERA, THERMOSTAT, LOCK, HEALTH, HOME, ISP, MESH, TWO_WAY, MILITARY, PHONE, OTHER. BODYWORN remains in the registry so older files parse. Fieldwatch displays it with wearables.

Decode sources. manufacturerData, serviceData.

Decode types. u8, i8, u16, i16, u24, u32, i32, f32, bits, utf8, hex, mac, bool.

Decode endian. le, be.

Decode gate operations. eq, neq, mask, nmask, len.

Publisher token reserved by this project. fieldwatch.

References

1. S. Bradner, Key words for use in RFCs to Indicate Requirement Levels, BCP 14, RFC 2119.

2. B. Leiba, Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words, BCP 14, RFC 8174.

3. T. Bray, The JavaScript Object Notation (JSON) Data Interchange Format, RFC 8259.

4. IEEE, Guidelines for Use of Extended Unique Identifier (EUI), Organizationally Unique Identifier (OUI), and Company ID.

5. Bluetooth Core Specification, Service UUID and the Bluetooth base UUID. Bluetooth SIG, Assigned Numbers, Company Identifiers.

6. Fieldwatch stock file fieldwatch-signatures, formatVersion 1, as published by Off Grid Pete LLC. Catalog 113 is the stock snapshot current with Proposal 1.

Comments and proposed profiles are invited at github.com/OffGridPete/Fieldwatch. Section 26 is how a project takes part. This revision is Proposal 1, 14 November 2026. Fieldwatch™ is a trademark of Off Grid Pete LLC.